COURSE CODE/TITLE:	CPE 031 - VISUALIZATIONS AND DATA ANALYSIS	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 17, 2026	**DATE SUBMITTED:**	Oct 24, 2026
NAME:	Mendoza, Roy Gabriel I. Mendoza, Ryo Shin P. Obias, Jave Joaquin C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12 **[Data Dashboard Taxonomies]**

1. PROCEDURE OUTPUT

Task 1:
Principles of Effective Data Dashboards

Code

```
# Task 1 Code Here
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Student performance dataset
data = {
    "Student": ["Ana", "Ben", "Carlo", "Dina", "Ethan",
                "Faith", "Gab", "Hana", "Ivan", "Jade"],
    "Subject": ["Math", "Science", "English", "Math", "Science",
                "English", "Math", "Science", "English", "Math"],
    "Score": [92, 85, 88, 78, 90, 95, 82, 76, 89, 94],
    "Attendance": [95, 90, 88, 85, 96, 98, 87, 80, 92, 97]
}

df = pd.DataFrame(data)

# Calculate average score per subject
subject_avg = df.groupby("Subject", as_index=False)["Score"].mean()

# Calculate average attendance per subject
attendance_avg = df.groupby("Subject", as_index=False)["Attendance"].mean()

# Create dashboard layout
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=(
        "Student Scores",
        "Average Score by Subject",
        "Average Attendance by Subject",
        "Score Distribution"
    ),
    specs=[
        [{"type": "bar"}, {"type": "bar"}],
        [{"type": "bar"}, {"type": "histogram"}]
    ]
)
```

```python
# Student scores
fig.add_trace(
    go.Bar(
        x=df["Student"],
        y=df["Score"],
        name="Score",
        marker_color="royalblue"
    ),
    row=1, col=1
)

# Average score by subject
fig.add_trace(
    go.Bar(
        x=subject_avg["Subject"],
        y=subject_avg["Score"],
        name="Average Score",
        marker_color="mediumseagreen"
    ),
    row=1, col=2
)

# Average attendance by subject
fig.add_trace(
    go.Bar(
        x=attendance_avg["Subject"],
        y=attendance_avg["Attendance"],
        name="Attendance",
        marker_color="orange"
    ),
    row=2, col=1
)
```

```python
# Distribution of student scores
fig.add_trace(
    go.Histogram(
        x=df["Score"],
        name="Score Distribution",
        marker_color="mediumpurple",
        nbinsx=5
    ),
    row=2, col=2
)

# Improve readability
fig.update_layout(
    title_text="Student Performance Dashboard",
    height=700,
    width=1100,
    template="plotly_white",
    showlegend=False
)

fig.update_yaxes(title_text="Score", row=1, col=1, range=[0, 100])
fig.update_yaxes(title_text="Average Score", row=1, col=2, range=[0, 100])
fig.update_yaxes(title_text="Attendance (%)", row=2, col=1, range=[0, 100])
fig.update_xaxes(title_text="Score", row=2, col=2)
fig.update_yaxes(title_text="Number of Students", row=2, col=2)

fig.show()
```

Output



ANALYSIS

Mendoza, Roy Gabriel	The output shows the visual encoding principles that makes it easier to track the academic performance and the attendance correlations at a quick look at it. I also learned how to make it in python where the information would show when hovering over the document or element in the visual output.
Mendoza, Ryo Shin	The output demonstrates how visual encoding principles make it easier to quickly understand student performance and the relationship between grades and attendance. I also learned how to use Python to add interactive features that display additional information when hovering over elements in the visualizations.
Obias, Jave Joaquin	In this task, the output displays visual encoding principles that make it easier to understand student performance and the relationship between grades and attendance. Also, I learned how to add interactive features that can show information when hovering documents or elements in the visual output using python.

Task 2:
Data Dashboard Taxonomies

Code

```python
#Code Here
import ipywidgets as widgets
from IPython.display import display, clear_output
import time

# 1. Interactive Dropdown and Slider widgets
region_dropdown = widgets.Dropdown(
    options=["All", "North", "South", "East", "West"],
    value="All",
    description="Region:",
    style={"description_width": "initial"}
)

sales_slider = widgets.IntSlider(
    value=5000,
    min=5000,
    max=20000,
    step=1000,
    description="Min Sales:",
    style={"description_width": "initial"}
)

output_area = widgets.Output()

# Function to update the chart based on widget inputs
def update_dashboard(change=None):
    with output_area:
        clear_output(wait=True)

        # Filter dataframe based on widgets
        filtered_df = df.copy()
        if region_dropdown.value != "All":
            filtered_df = filtered_df[filtered_df["Region"] == region_dropdown.value]
        filtered_df = filtered_df[filtered_df["Sales"] >= sales_slider.value]

        if filtered_df.empty:
            print("No data matches the selected filters.")
            return

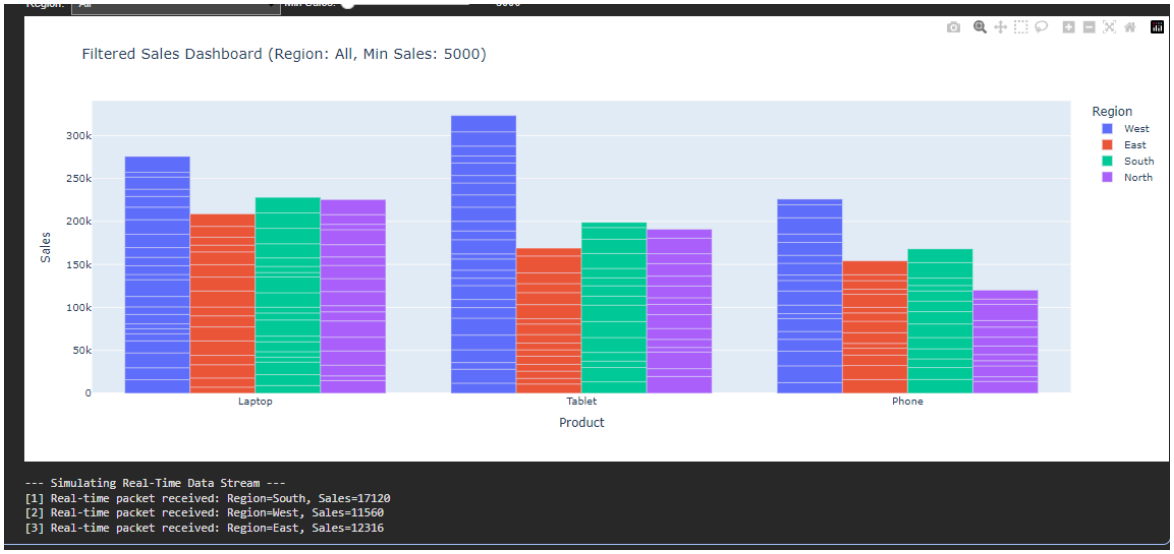
        # Generate Plotly chart
        fig = px.bar(
            filtered_df,
            x="Product",
            y="Sales",
            color="Region",
            barmode="group",
            title=f"Filtered Sales Dashboard (Region: {region_dropdown.value}, Min Sales: {sales_slider.value})"
        )
        fig.show()

# Bind widgets to the update function
region_dropdown.observe(update_dashboard, names="value")
sales_slider.observe(update_dashboard, names="value")

# Display controls and initial output
display(widgets.VBox([widgets.HBox([region_dropdown, sales_slider]), output_area]))
update_dashboard()

# 2. Simulated Real-Time Dynamic Update Function
print("\nSimulating Real-Time Data Stream")
for i in range(3):
    # Simulate incoming real-time sales record
    new_row = pd.DataFrame({
        "Month": [pd.Timestamp.now()],
        "Region": [np.random.choice(["North", "South", "East", "West"])],
        "Product": [np.random.choice(["Laptop", "Tablet", "Phone"])],
        "Sales": [np.randint(6000, 18000) if hasattr(np, 'randint') else np.random.randint(6000, 18000)],
        "Profit": [np.random.randint(600, 4500)]
    })
    # Append to live stream dataset simulation
    print(f"[{i+1}] Real-time packet received: Region={new_row['Region'].values[0]}, Sales={new_row['Sales'].values[0]}")
    time.sleep(1) # Wait 1 second to simulate live feed interval
```

Output



ANALYSIS	
Mendoza, Roy Gabriel	Creating the Task 2 made me understand how to make a dynamic dashboard that can be interacted with by the users like disable or enable a dropdown or slider to hide or release information.
Mendoza, Ryo Shin	Working on Task 2 taught me how to build an interactive dashboard where users can control the displayed data using features such as dropdown menus and sliders to filter information and customize what they see.
Obias, Jave Joaquin	The Task 2 helps me to make an interactive dashboard that can display data with some features like dropdown or slider that can customize or filter information.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary:

Create a mini dashboard project that tracks student grades or attendance:

1. Include 3 visualizations (e.g., average grades by subject, attendance by week, and performance trend).
2. Add at least one interactive feature (dropdown or slider).
3. Explain how your dashboard helps an instructor or department make data-driven decisions.

Code

```python
# CODE

import pandas as pd
import plotly.express as px
from plotly.subplots import make_subplots
import plotly.graph_objects as go
import ipywidgets as widgets
from IPython.display import display, clear_output

# 1. Dataset Generation for Mini Project (Student Grades & Attendance)
data_mini = {
    "Student": ["Alice", "Bob", "Charlie", "Diana", "Ethan", "Fiona", "George", "Hannah"],
    "Subject": ["Math", "Math", "Science", "Science", "English", "English", "Math", "Science"],
    "Grade": [88, 76, 92, 85, 90, 78, 95, 82],
    "Attendance_Pct": [95, 82, 98, 90, 96, 75, 99, 88],
    "Week": ["Week 1", "Week 2", "Week 3", "Week 4", "Week 1", "Week 2", "Week 3", "Week 4"]
}
df_mini = pd.DataFrame(data_mini)

# 2. Interactive Widget (Subject Dropdown)
subject_dropdown = widgets.Dropdown(
    options=["All", "Math", "Science", "English"],
    value="All",
    description="Subject Filter:",
    style={"description_width": "initial"}
)

output_widget = widgets.Output()

def render_mini_dashboard(change=None):
    with output_widget:
        clear_output(wait=True)

        # Filter dataset based on selection
        filtered = df_mini.copy()
        if subject_dropdown.value != "All":
            filtered = filtered[filtered["Subject"] == subject_dropdown.value]

        if filtered.empty:
            print("No records found for this selection.")
            return

        # Create 3 Visualizations using Subplots with clean spacing
        fig = make_subplots(
            rows=1, cols=3,
            subplot_titles=("Grades by Student", "Attendance vs Grade", "Weekly Distribution"),
            horizontal_spacing=0.12
        )
```

```
        # Visualization 1: Grades by Student Bar Chart
        fig.add_trace(
            go.Bar(
                x=filtered["Student"],
                y=filtered["Grade"],
                name="Grades",
                marker_color="#2b5c8f",
                text=filtered["Grade"],
                textposition="outside"
            ),
            row=1, col=1
        )

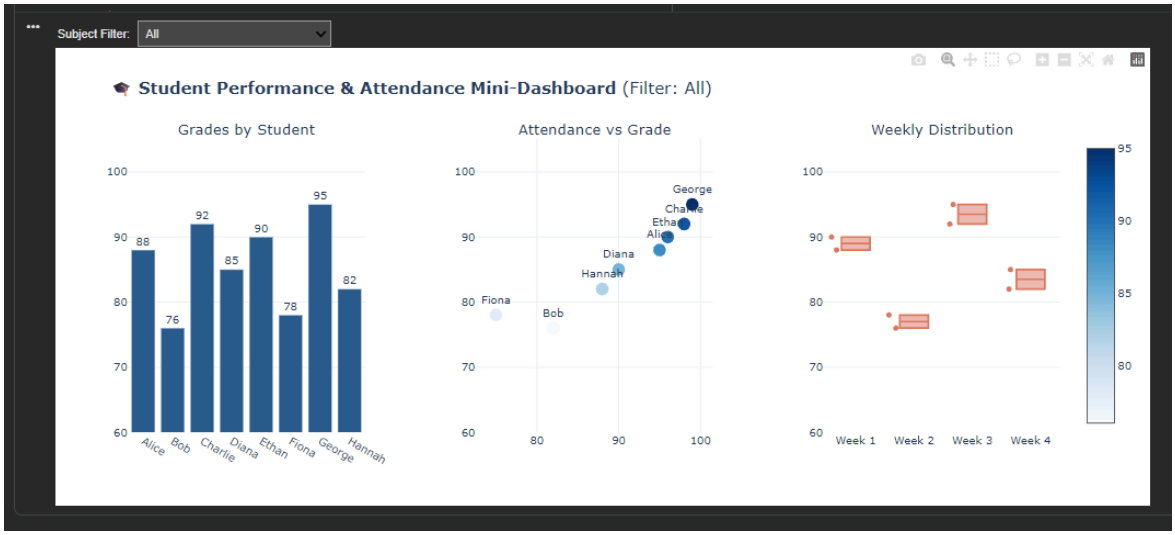
        # Visualization 2: Attendance vs Grade Scatter Plot
        fig.add_trace(
            go.Scatter(
                x=filtered["Attendance_Pct"],
                y=filtered["Grade"],
                mode="markers+text",
                text=filtered["Student"],
                textposition="top center",
                marker=dict(size=14, color=filtered["Grade"],
                colorscale="Blues", showscale=True),
                name="Correlation"
            ),
            row=1, col=2
        )

        # Visualization 3: Weekly Grade Box/Spread Chart
        fig.add_trace(
            go.Box(
                x=filtered["Week"],
                y=filtered["Grade"],
                name="Weekly Spread",
                marker_color="#e07a5f",
                boxpoints="all"
            ),
            row=1, col=3
        )

        # Layout styling refinements
        fig.update_layout(
            height=500,
            width=1200,
            title_text=f"<b>📊 Student Performance & Attendance Mini-Dashboard</
            b> (Filter: {subject_dropdown.value})",
            title_font_size=18,
            showlegend=False,
            template="plotly_white"
        )

        # Axis cleanup
        fig.update_yaxes(range=[60, 105])
        fig.show()

# Bind and display
subject_dropdown.observe(render_mini_dashboard, names="value")
display(widgets.VBox([subject_dropdown, output_widget]))
render_mini_dashboard()
```

Output


Analysis	
Mendoza, Roy Gabriel	In this supplementary activity, we have applied the past lessons and made the overall dashboard be concise but still showing all the necessary information. Additionally we have applied the lessons in the procedure, on how to like add the dynamic dashboard , and utilize the advance plotly functions or parameters like markers and color scales, with plotly.graphs and ipywidgets resulting to the making of the visual output where we can easily identify the correlation of the attendance and the grades from the general filter, subject filter: All.
Mendoza, Ryo Shin	Through this supplementary activity, we combined what we learned from previous lessons to develop a simple yet informative dashboard. We explored how Plotly features, including markers and color scales, can improve data presentation, while ipywidgets allows users to filter information interactively. By combining these tools, we created visualizations that help us better understand the connection between student attendance and grades when all subjects are displayed.
Obias, Jave Joaquin	In this supplementary part of the activity, we used what we learned in the past lesson that can improve a dashboard. We used plotly functions like markers and color scales to develop data visuals, we also used ipywidgets to filter information so that we can easily understand the relationship between student attendance and grades when displaying all the subjects.

Conclusion	
Mendoza, Roy Gabriel	Overall, this activity provided and showed me the ways to make a comprehensive and interactive dashboard with dynamic and interactive

	implementations. Task 1 highlighted the organization and the visual hierarchy, while task 2 is where we were able to demonstrate how widgets and other interactive elements are used in a dashboard. Lastly the Supplementary activity made us integrate all of the contents of the past lessons and the procedure part of this activity
Mendoza, Ryo Shin	This activity helped me develop my skills in creating dashboards that present data clearly and allow users to interact with the visualizations. In Task 1, I learned the importance of arranging information properly to make it easier to understand. Task 2 introduced me to interactive tools that let users filter and explore data based on their needs. Finally, the supplementary activity allowed me to combine the techniques from previous lessons and apply them in creating a functional and informative dashboard.
Obias, Jave Joaquin	Overall, in this activity, it helps me understand how to create dynamic dashboards using Python. Tasks 1 and 2 highlighted the importance of visual encoding principles and interactive controls like hover features, dropdowns, and the sliders for filtering and analyzing key information. In conclusion, applying these techniques in the supplementary activity allowed us to combine plotly visual features with ipywidgets to easily understand the relationship between student grades and attendance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.